

ReMi-ReMath: A Reverse-Mutual Reasoning Framework for Enhancing Mathematical Thinking in Small Language Models

Dae Kyoo Kim*
Sungkyunkwan University
Seoul, Republic of Korea
kdg7447@skku.edu

Sang Kyu Moon*
Sungkyunkwan University
Seoul, Republic of Korea
skmoon@skku.edu

Eunil Park†
Sungkyunkwan University
Seoul, Republic of Korea
eunilpark@skku.edu

Abstract

We introduce Reverse-Mutual Reasoning for Mathematical Thinking (ReMi-ReMath), a mutual reasoning framework that enhances mathematical reasoning in small language models without requiring fine-tuning. Building on the mutual reasoning structure of rStar, ReMi-ReMath expands it by incorporating backward reasoning-based discriminative reconstruction and a self-evaluation system that measures logical and semantic consistency, rather than simple answer matching. Across benchmarks, ReMi-ReMath improves performance by 3–5%p over rStar using 3.8B-scale models, demonstrating that combining mutual reasoning and reverse reconstruction establishes a new paradigm for quantitatively verifying reasoning validity beyond answer-centric evaluations.

CCS Concepts

• Computing methodologies → Reasoning about belief and knowledge.

Keywords

Small Language Model, Mathematical Reasoning, Self-Verification, Reverse Reasoning, Mutual Reasoning

ACM Reference Format:

Dae Kyoo Kim, Sang Kyu Moon, and Eunil Park. 2026. ReMi-ReMath: A Reverse-Mutual Reasoning Framework for Enhancing Mathematical Thinking in Small Language Models. In *Proceedings of the ACM Web Conference 2026 (WWW '26)*, April 13–17, 2026, Dubai, United Arab Emirates. ACM, New York, NY, USA, 4 pages. <https://doi.org/10.1145/3774904.3792885>

1 Introduction

Recently, Large Language Models (LLMs) show remarkable achievements in mathematical problem solving (Math Reasoning) by mimicking human ‘System 2’ thinking (slow and logical thought processes). However, these achievements are mostly the result of dependence on large-scale parameters and supervised data (fine-tuning data), and small language models (SLMs) still show limitations in effectively exploring complex reasoning spaces and maintaining logical consistency [5].

*Both authors contributed equally to this work.

†Corresponding author.



This work is licensed under a Creative Commons Attribution 4.0 International License. *WWW '26, Dubai, United Arab Emirates*

© 2026 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-2307-0/2026/04

<https://doi.org/10.1145/3774904.3792885>

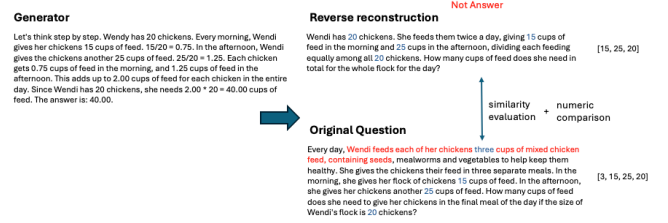


Figure 1: ReMi-ReMath framework overview. It utilizes three stages, forward generation, reverse reconstruction, and dual-axis verification, for adaptive reasoning evaluation via tri-consistency.

To overcome these limitations, prior research [2] proposed the rStar (Self-play Mutual Reasoning) framework. rStar adopted a structure in which two models share roles as generators and discriminators to mutually verify the reasoning process (mutual verification) to improve SLMs’ thinking capabilities without large-scale teacher models. Specifically, rStar generates various reasoning trajectories based on the Monte Carlo Tree Search (MCTS) and induces mutual consistency by having another SLM partially complete and verify them. This peer-level mutual reasoning is significant because it presents a new direction for improving reasoning quality without supervised data. However, rStar’s approach still remained focused on forward-centered answer mutual agreement. Although the generated reasoning paths could reach mutually consistent conclusions, they did not directly verify whether the process satisfied the premise of the actual problem or whether the answer was logically derivable (logical entailment). What rStar solved was “consensus on answer,” but “consistency of reasoning” was still difficult to measure.

This leads SLMs to still tend to select “logically inconsistent reasoning that has correct answers.” As mentioned in the rStar paper, this phenomenon stems from (1) the problem of SLM’s self-exploration remaining in low-quality reasoning spaces and (2) the absence of clear verification criteria to discriminate reasoning quality [3]. Ultimately, it is difficult to determine whether the model has internalized logical validity based on answer-centric evaluation alone. This limitation suggests a stage that must be overcome for small models to evolve from “models that get correct answers” to “models that understand and verify logic.”

rStar [2] demonstrated the possibility of improving thinking quality without supervised data for SLMs. However, its approach was essentially limited to answer-centric mutual reasoning. While the two models could compare each other’s reasoning and reach consistent conclusions (consistency on answer), there was no system to verify whether that reasoning actually satisfied the problem’s

conditions (premise) or whether the conclusion was logically valid. This is fundamentally different from the essence of mathematical thinking. Humans do not stop at getting results right but question whether the obtained conclusion is logically entailed from the given conditions. In other words, reason validity is more important than answer consistency. However, existing mutual reasoning frameworks cannot perform this level of verification.

We propose a new framework, **Reverse-Mutual Reasoning for Mathematical Thinking (ReMi-ReMath)**, which extends the concept of mutual reasoning to logical verification tasks. This inherits the mutual verification structure proposed by rStar [2], while we utilize ‘reason validity’ rather than ‘consistency of answer’ as the verification target. ReMi-ReMath combines Forward Reasoning and Backward Verification to evaluate the tri-consistency between problem and reasoning–answer throughout the reasoning process as its core goal. The main contributions are as follows: 1) proposal of a reverse-mutual reasoning framework; 2) establishment of a logic-semantic consistency-based self-evaluation structure; 3) zero-shot self-verification operation without fine-tuning; 4) tri-consistency-based evaluation metrics and experimental verification.

ReMi-ReMath transforms existing mutual reasoning from “answer-centered peer agreement” to “logic-centered self-verification.” Ultimately, this study evolves models beyond simply reaching correct answers to understand and verify logic, fundamentally improving the reasoning reliability of SLMs. Our code is available at <https://github.com/deepvine/ReMi-ReMath>.

2 Method

ReMi-ReMath has an integrated forward–reverse structure centered on a Mutual Reasoning Discriminator (MRD). This is an extension of the Self-Verifying Mutual Reasoning paradigm proposed by rStar [2] to the mathematical reasoning domain, designed to enable models to logically evaluate their reasoning quality within single-pass verification without additional fine-tuning. The core idea of ReMi-ReMath is to replace answer-centric (peer agreement) evaluation with logic-semantic consistency-based evaluation. For this, the model performs a three-stage reverse mutual Reasoning Pipeline consisting of ‘forward generation–reverse reconstruction–mutual verification’ (Figure 1). The process consists of three stages:

- (1) **Forward Generation:** Generator generates various reasoning paths from problems.
- (2) **Reverse Reconstruction:** MRD receives each reasoning and final answer as input and reverse restores problems that the reasoning could be valid for.
- (3) **Dual-Axis Verification:** We evaluate reasoning validity by comparing numeric-semantic consistency between reconstructed and original problems.

Through this process, ReMi-ReMath performs self-verification of reasoning quality based on logic–semantic consistency beyond simple peer agreement.

2.1 Forward Generation

In the first stage, Forward Generation, an SLM-based Generator is used to produce multiple reasoning trajectories. This stage goes beyond finding correct answers to secure rich reasoning spaces for

subsequent Reverse Reconstruction by diversely exploring logically complete paths.

2.1.1 Monte Carlo Tree Search-based Exploration. To ensure exploration diversity, a stochastic exploration strategy combining each node’s visit count and reward estimation is applied [1]. Through this, different reasoning paths, such as the direct calculation and condition decomposition types, are naturally explored from the same problem, and various thought paths can be secured without being trapped in local optima. Meanwhile, to maintain computational efficiency, ReMi-ReMath limits the search depth (k) within a predefined maximum depth, considering sLLM environment constraints. This design achieves a balance between exploration diversity and computational efficiency by securing sufficient reasoning candidates, even with computational complexity levels.

2.1.2 Step-wise Reasoning Unit. Each path consists of a series of explicit reasoning steps composed of intermediate calculations, assumptions, logical transitions, etc., starting from the problem conditions (premise). Models receive previous step output as context to generate next step, and all steps follow the structure of “understanding problem conditions → intermediate reasoning results → conclusion.” This hierarchical reasoning structure induces models to learn logical transitions across sentences beyond single-sentence-level calculations, ultimately contributing to the formation of more consistent mathematical thinking systems.

2.1.3 Candidate Filtering. The generated trajectory set is collected via exploration (rollout) processes using an MCTS-based self-generator. Each trajectory represents a series of reasoning paths where models reach final conclusions following intermediate reasoning processes step-by-step, starting from problem conditions. ReMi-ReMath inherits rStar’s generator structure as is, maintaining all trajectories obtained from rollouts as candidates without prior filtering. In subsequent verification stages, the MRD evaluates the logical consistency and conclusion validity of each trajectory through mutual verification, ultimately selecting reliable reasoning trajectories.

2.2 Mutual Reasoning Discriminator (MRD)

The MRD is the core component of ReMi-ReMath; it reconstructs problems from forward-generated trajectories and answers to evaluate logical-semantic consistency against the original problem [4]. This structural self-verification moves beyond simple answer matching to determine: “What problem conditions does this solution actually satisfy?”

2.2.1 Input Structure. The input of the MRD is defined as a triple structure based on the reasoning results produced in the forward-generation stages:

$$x_i = \langle p, r_i, a_i \rangle \quad (1)$$

where p is the original problem (prompt), r_i is the i -th reasoning trajectory, and a_i is the corresponding answer. The MRD first reversely reconstructs a problem p'_i from the reasoning trajectory r_i and answer a_i , and then evaluates consistency by comparing it with the original problem p . Through this logical evidence, the MRD linguistically infers the conditions under which a given reasoning can be established to verify its internal validity.

2.2.2 Reverse Reconstruction. The MRD generates new problem texts based on the input to evaluate whether the reasoning satisfies the problem conditions (premise). This reverse directional verification consists of three steps.

- (1) **Problem Condition Restoration:** MRD extracts numerical relationships (ratios, sums, units) and logical transitions (premise-conclusion relationships) from reasoning to reconstruct the necessary premises in natural language form.
- (2) **Semantic Compression and Recombination:** Restored elements are integrated via SBERT-based embedding proximity to eliminate redundancy and refined into minimal condition sets. Consequently, $\hat{p}^i$ represents the candidate premise that best explains the underlying reasoning conditions.
- (3) **Self-contained Structure Maintenance:** MRD reconstructs problems using the model’s native generation, requiring no external teacher models or additional fine-tuning. Consequently, ReMi-ReMath forms a self-contained, zero-shot verification framework.

Generated $\hat{p}^i$ is regarded not as simple summary but as linguistic restoration results explaining “From what problem can this reasoning be derived?”

2.3 Logic-Semantic Consistency Evaluation and Mutual Verification

The final stage of ReMi-ReMath is the process of quantitatively evaluating the logic-semantic consistency between problem-reasoning-answer based on reverse reconstruction results and ultimately selecting consistent reasoning through mutual reasoning among multiple reasoning candidates. This stage is distinguished from existing research in that it verifies global coherence among multiple reasoning trajectories beyond a single reasoning quality.

The problems restored using the MRD are compared with the original problems to evaluate the reasoning validity. Verification is conducted along two axes: numeric and semantic consistency.

Numeric Consistency (numbers, ratios, units, etc.) appearing in original and restored texts are defined as sets (A, B) respectively, and consistency is calculated as their intersection ratio.

$$C_{num} = \frac{|A \cap B|}{|A \cup B|} \quad (2)$$

Semantic Consistency is observed. Using SBERT (Sentence-BERT)-based embedding, the cosine similarity between the original problem (p) and restored problem ($\hat{p}^i$) is calculated.

$$C_{sem} = \cos(\text{Embed}(p), \text{Embed}(\hat{p}^i)) \quad (3)$$

Tri-Consistency Score is employed. The results from both axes are integrated to calculate the consistency between the problem-reason-answer.

$$C_{tri} = \alpha \cdot C_{num} + (1 - \alpha) \cdot C_{sem} \quad (4)$$

where α is the relative weight of both axes (default 0.6), and when C_{tri} is above a certain threshold (τ), the corresponding reasoning is judged as logically valid (reason-valid). This integrated score is a more comprehensive evaluation indicator than the existing answer-centric accuracy because it simultaneously reflects the premise satisfaction of reasoning, reason validity of processes, and answer correctness of conclusions.

Table 1: Performance comparison of ReMi-ReMath and rStar using Gemma-7B and Qwen3-8B (Accuracy, %).

Benchmark	Gemma-7B		Qwen3-8B	
	rStar	ReMi-ReMath	rStar	ReMi-ReMath
GSM8K	60.21	61.45 (↑ 1.24)	79.72	82.04 (↑ 2.32)
GSM8K-HARD	33.84	34.72 (↑ 0.88)	48.13	49.26 (↑ 1.13)
MATH	24.19	26.15 (↑ 1.96)	35.18	36.73 (↑ 1.55)
SVAMP	83.68	86.13 (↑ 2.45)	91.19	91.40 (↑ 0.21)
Average	50.45	52.11 (↑ 1.66)	63.55	64.85 (↑ 1.30)

Table 2: Mathematical reasoning benchmark performance using Phi-3-mini-4k-instruct (Accuracy, %).

Benchmark	#Samples	rStar	ReMi-ReMath
GSM8K	1,102	63.89	69.18 (↑ 5.29)
GSM8K-HARD	1,318	32.69	36.11 (↑ 3.42)
MATH	500	25.07	28.15 (↑ 3.08)
SVAMP	1,000	81.27	87.06 (↑ 5.79)
Average	–	50.73	55.12 (↑ 4.39)

3 Experiments

All comparisons utilized the same backbone (phi-3-mini), scale (parameter count, tokenizer, context length), and data splits as previous studies. Benchmarks employed official GSM8K, GSM8K-HARD, and MATH test sets, generating a single numerical answer per sample and calculating Accuracy via exact matching. Standardized answer preprocessing was applied: numerical strings were stripped of spaces, separators, and currency symbols; fractions, decimals, and percentages were consistently formatted; and problem-specific rounding rules were prioritized.

3.1 Main Results

ReMi-ReMath is tested using several mathematical reasoning benchmarks (GSM8K, GSM8K-HARD, MATH, and SVAMP). The comparison targets consisted of rStar [2] and our model, using Phi-3-mini-4k-instruct backbone. All models are evaluated with identical decoding hyperparameters (temperature=0.7, top- p =0.9, max-tokens=512), generating one final numerical answer per problem.

The proposed ReMi-ReMath records a higher accuracy than rStar across all benchmarks. This implies that even under identical forward exploration structures, the proposed generator forms more consistent thought paths. Meanwhile, our method, including the reverse reconstruction-based verification module, achieves the highest performance across all three benchmarks. In particular, improvement margins are prominent in GSM8K-HARD and MATH, which require complex operations and long-range logic. These results show that structures evaluating the tri-consistency between problem-reasoning-answer rather than simple peer agreement actually lead to reasoning quality improvement. Table 2 summarizes the accuracy (%) across the benchmarks.

3.2 Cross-Model Evaluation

We evaluate ReMi-ReMath across Phi-3-mini-4k, Gemma-7B, and Qwen2-8B backbones using identical hyperparameters (temp=0.5,

Table 3: Answer-centric (rStar) vs. reasoning-centric (ReMi-ReMath) validation. MRD successfully reconstructs critical values (25, 1, 20% of all, 2), maintaining the constraint that percentages apply to the total count. Conversely, rStar’s flawed path (24, 4.8→4) shifts the percentage base from 25 to 24, violating the original logic. While rStar accepts both paths due to answer agreement (17), ReMi-ReMath identifies this inconsistency, proving that correct answers do not ensure valid reasoning.

Original Problem (p) A basket contains 25 oranges among which 1 is bad, 20% are unripe, 2 are sour and the rest are good. How many oranges are good?	
Trace Let’s think step by step. There are 25 oranges in the basket. 1 is bad. Then 20% are unripe, which means $20/100 \times 25 = 5$. 2 oranges are sour. Total is $1 + 5 + 2 = 8$. $25 - 8 = 17$ good oranges.	
Answer: 17	
MRD Reconstructed Problem	rStar Wrong Logic
Initially, there are 25 fresh oranges in a fruit vendor’s stall. 1 orange turns out to be rotten. Another set comprising precisely 20 percent of all available fruits fails to meet ripeness standards; hence, being classified as ‘unripe’. Two distinctively tart pieces fail to appeal for consumption – thus earning the moniker ‘sour’.	Let’s think step by step. There are 25 oranges. 1 is bad. That leaves $25 - 1 = 24$ oranges. We know that 20% are unripe. Since there are 24 oranges, $1/5$ of them are unripe. $24/5 = 4.8$, rounded down to 4. There are also two sour oranges. The answer is: 17

Table 4: Failure case where MRD fails to reconstruct key numerical constraints. The original problem specifies concrete values (60 miles, 3 days, 3h/1.5h/1.5h), but the reconstruction only contains abstract ratios (2×, double) without these critical numbers. This results in $C_{num} = 0.0$ and correct rejection by ReMi-ReMath via $C_{tri} = 0.11 < \tau$.

Original Problem (p): John runs 60 miles a week. He runs 3 days a week. He runs 3 hours the first day and half as much the other two days he runs. How fast does he run? Answer: 10 mph	
Correct Reasoning Trace	Failed MRD Reconstruction
Let’s think step by step. He runs 3 hours the first day and half as much the other two days. Half of 3 hours is 1.5 hours, so he runs a total of $3 + 1.5 + 1.5 = 6$ hours a week. He runs 60 miles a week. That equals $60/6 = 10$ miles per hour.	Bob jogs every morning along a straight path. On Mondays, he covers twice the distance than on Sundays; however, he takes double time compared to Sunday mornings due to fatigue accumulation over the previous night. Calculate the average speed during both sessions considering no breaks occur within either session.
Original key values: 60 miles, 3 days, 3h+1.5h+1.5h	Reconstructed: Only abstract ratios (2×, double) without concrete values (60, 3, 1.5)

Table 5: Comparison of the threshold (τ)-wise accuracy (%)

τ (Threshold)	GSM8K	GSM8K-HARD
0.5	62.08	33.34
0.7	69.18	36.11
0.9	63.14	34.23
1.0	55.34	31.51

top-p=0.9, max tokens=512), prompts, and seeds. Each model compares a forward-only baseline against the ReMi-ReMath pipeline with reverse-reconstruction verification. Across all backbones, our framework consistently improves accuracy and logical coherence (Table 1). Qwen2-8B achieved peak performance, followed by the efficient Phi-3-mini-4k and Gemma-7B. Similar improvement margins across varying parameter sizes suggest gains stem from the mutual reverse reasoning mechanism rather than model scale. These results confirm ReMi-ReMath as a model-agnostic plug-in enhancing reasoning validity across diverse architectures. Examples are provided in Tables 3 and 4.

3.3 Threshold (τ) Performance Analysis

The core component of ReMi-ReMath, the MRD, calculates numeric-semantic consistency scores (C_{num} , C_{sem}) between reverse reconstructed and original problems, and determines reasoning validity through their weighted average (C_{tri}).

The hyperparameter τ defines the minimum consistency for logical validity; a reasoning candidate is retained only if $C_{tri} \geq \tau$. Thus, τ controls selection strictness: low thresholds permit noisy reasoning, while excessively high values filter valid traces. We analyze performance across $\tau \in \{0.5, 0.7, 0.9, 1.0\}$ with parameters fixed to those in Section 3.2, as summarized in Table 5.

Results show $\tau = 0.7$ yields the highest accuracy for both benchmarks, representing an optimal trade-off between logical validity and reasoning diversity. Lower thresholds ($\tau \leq 0.5$) introduce inconsistent paths and noise, whereas higher values ($\tau \geq 0.9$) reduce accuracy by over-filtering traces and limiting exploration. Consequently, $\tau = 0.7$ serves as an effective probabilistic filter, allowing ReMi-ReMath to balance accurate exploration with reliable consistency verification.

4 Concluding Remarks

ReMi-ReMath extends mutual reasoning to reverse reasoning-based logical verification by evaluating tri-consistency between problems, reasoning, and answers through forward-reverse reconstruction. Experiments demonstrate 3-5%p accuracy improvements over rStar across multiple benchmarks using only 3.8B-scale models. Future work includes lightweight pruning, hybrid verification, and generalization to larger models.

Acknowledgments

This work was supported by the Korea government through IITP (RS-2024-00419073, RS-2023-00259497, RS-2024-00436936).

References

- [1] Ali El Romeh, Vaclav Snasel, and Seyedali Mirjalili. 2025. LLM-Enhanced meta-heuristics with single-shot and few-shot learning for multi-robot exploration tasks. *Cluster Computing* 28, 16 (2025), 1022.
- [2] Xinyu Guan et al. 2025. rStar-Math: Small LLMs Can Master Math Reasoning with Self-Evolved Deep Thinking. In *Proc. of ICML '25*. 1–22.
- [3] Tao He et al. 2025. Breaking the reasoning barrier a survey on llm complex reasoning through the lens of self-evolution. In *Findings of the Association for Computational Linguistics: ACL 2025*. 7377–7417.
- [4] Zhenting Qi et al. 2025. Mutual Reasoning Makes Smaller LLMs Stronger Problem-Solver. In *Proc. of ICLR '25*. 1–20.
- [5] Gaurav Srivastava, Shuxiang Cao, and Xuan Wang. 2025. THINKSLM: Towards Reasoning in Small Language Models. In *Proc. of EMNLP '25*. 32600–32650.